
Universality and Limitations of Prompt Tuning

Yihan Wang
UCLA

wangyihan617@gmail.com

Jatin Chauhan
UCLA

chauhanjatin100@gmail.com

Wei Wang
UCLA

weiwang@cs.ucla.edu

Cho-Jui Hsieh
Google and UCLA
chohsieh@cs.ucla.edu

Abstract

Despite the demonstrated empirical efficacy of prompt tuning to adapt a pretrained language model for a new task, the theoretical underpinnings of the difference between "tuning parameters before the input" against "the tuning of model weights" are limited. We thus take one of the first steps to understand the role of soft-prompt tuning for transformer-based architectures. By considering a general purpose architecture, we analyze prompt tuning from the lens of both: universal approximation and limitations with finite-depth fixed-weight pretrained transformers for continuous-valued functions. Our universality result guarantees the existence of a strong transformer with a prompt to approximate any sequence-to-sequence function in the set of Lipschitz functions. The limitations of prompt tuning for limited-depth transformers are first proved by constructing a set of datasets, that cannot be memorized by a prompt of any length for a given single encoder layer. We also provide a lower bound on the required number of tunable prompt parameters and compare the result with the number of parameters required for a low-rank update (based on LoRA) for a single-layer setting. We finally extend our analysis to multi-layer settings by providing sufficient conditions under which the transformer can at best learn datasets from invertible functions only. Our theoretical claims are also corroborated by empirical results.

1 Introduction

The surge in the empirical research of large-scale models has led to the emergence of a new paradigm of prompt tuning. Current large models consist of billions of parameters [Brown et al., 2020, Chowdhery et al., 2022], which greatly exacerbate the cost of tuning the entire model weights via gradient-based optimization. On the other hand, the power of scale in both model size and pretraining dataset size has demonstrated strong capabilities by achieving reasonable performance through a learnable prompt appended before the input [Li and Liang, 2021, Lester et al., 2021]. Despite this, several questions emanate around the abilities and limitations of prompt tuning.

In this work, we aim to characterize some natural yet essential questions about prompt tuning with transformer architectures. Firstly, are prompts universal approximators, i.e. with a fixed pretrained transformer network, can we find a prompt to approximate any sequence-to-sequence function in a given space? If yes, can we construct the transformer for this universality result? Second, can we identify failure modes of prompt tuning when applied on potentially non-optimal but non-trivial transformers? Moreover, since prompt tuning is usually compared against LoRA [Hu et al., 2021] in consideration to parameter-efficient tuning, is prompt tuning then more/less parameter-efficient than LoRA? Answering these questions can lead to important insights on *when* and *how* to perform prompt tuning to adapt a pretrained transformer network to a given downstream task of interest.

In this work, we seek to answer these questions with appropriate theoretical analysis and further validate our claims with empirical results. We first characterize the universal nature of prompt tuning by constructing a specific transformer network. We show that for a given approximation error and the space of sequence-to-sequence Lipschitz functions, we can construct a transformer network, with a suitable number of layers, that can leverage prompt tuning to approximate any function in this space. Despite this universality of prompt tuning with a carefully constructed pretrained transformer, we then identify some limitations of prompt tuning with weaker but non-trivial transformers. We prove this by constructing sequence-to-sequence datasets with shared input tokens, which are surprisingly simple but cannot be memorized by prompt tuning for a given transformer. We also extend our analysis to more general settings where the shared token is not required. In this setting, we first prove that prompt tuning on a single-layer transformer requires $\Omega(n)$ trainable parameters to memorize n training examples, wherein for LoRA, it suffices with $O(n)$ trainable parameters. We finally extend our analysis to the multi-layer setting and provide sufficient conditions under which prompt tuning exhibits extremely limited capacity to at best memorizing datasets from invertible functions.

Our contributions can be summarized as below:

- We characterize the universal nature of prompt tuning by explicitly constructing a transformer network (Theorem 1).
- We provide a construction-based argument for sequence-to-sequence datasets that cannot be learned by prompt tuning with a given single-layer transformer (Theorem 2).
- We provide the lower bound on the required number of parameters for prompt tuning to memorize any sequence-to-sequence functions (Theorem 3).
- We provide a sufficient condition for multi-layer transformers, under which datasets with shared output tokens cannot be learned with prompt tuning (Theorem 4).
- We conduct empirical studies, including real-world datasets, to verify our theoretical claims.

2 Related Work

Theoretical Analysis of Transformers Various works have characterized the theoretical properties of transformers and its primary self-attention component. [Yun et al., 2020] study the universal approximation ability of transformers for continuous permutation equivariant sequence-to-sequence functions with compact support and further examined the role of using positional encodings to circumvent permutation equivariant condition. [Pérez et al., 2021] show that transformer with a hard-attention is Turing complete based on their capacity to perform computations and access the internal dense representations of the data. [Wei et al., 2021] further show that transformers can approximate Turing machines with bounded computation time with a new notion of approximation. [Dong et al., 2021] provided a negative yet interesting result signifying the limitations of pure self-attention in terms of rank diminishing of the input. Other works including [Kim et al., 2021, Dasoulas et al., 2021] derive upper bounds on the Lipschitz constant of respective modifications of the attention mechanism. The works by [Li et al., 2022, Zhang et al., 2020] documented optimization perspective on transformer training via SGD.

Fine-tuning and Prompt Tuning Fine-tuning is the standard way to adapt a pretrained model to downstream tasks. The most standard and popular paradigm is tuning the model weights via a suitable optimization procedure along with a linear head on the output representations [Radford et al., 2018, Devlin et al., 2019]. Subsequent works studied more parameter-efficient ways of fine-tuning by updating either a subset of model parameters [Ben Zaken et al., 2022] or restricting the parameter-updates to a low-dimensional subspace [Aghajanyan et al., 2021, Hu et al., 2021, Mahabadi et al., 2021]. The work by [Hu et al., 2021] (their framework referred to as LoRA) has garnered particular interest in the community and [Malladi et al., 2023] has provided an interpretation of LoRA via the kernel mechanism. In the particular context of LLMs, prompt tuning has emerged as the de facto approach where only the prompt is updated while keeping the rest of the transformer weights and architecture fixed [Shin et al., 2020, Lester et al., 2021, Li and Liang, 2021].

Analysis of Prompt Tuning [Wei et al., 2022] studied the link between prompt tuning and downstream tasks with an underlying latent variable generative model of text, which is confined to a Hidden Markov Model. However, they focused on the discrete vocabulary setting contrary to our

results for continuous sequence-to-sequence functions. Some more recent works [Akyürek et al., 2023, Von Oswald et al., 2023] characterized an intriguing property of a specific form of prompting, referred to as in-context learning, where they proved by construction that transformers can implement learning algorithms for linear models based on gradient descent and closed-form ridge regression. This work however pursued a different and specific direction from the prompting results we aim to provide for generic settings.

Memorization Capacity of Neural Networks A series of works have sought to provide finite sample universal memorization capacity results of neural networks and the understanding of expressive power of neural networks. Huang and Huang [1990], Huang and Babri [1998], Huang [2003], Yamasaki [1993] analyzed the memorization capacity of FNNs with sigmoid and other bounded activation functions. Hardt and Ma [2016], Zhang et al. [2021], Nguyen and Hein [2018] provided results for modern ReLU networks including FNNs and CNNs. For transformer architectures, Kim et al. proved that transformers can memorize a dataset with finite parameters. To the best of our knowledge, similar results for prompt tuning have not been studied in continuous settings for transformer architectures.

3 Transformers and Parameter Efficient Training

3.1 Preliminaries

We use the following notations throughout the paper. A bold lower case character, e.g. $\mathbf{x}$, denotes a vector. A bold upper case character, e.g. $\mathbf{W}$, denotes a matrix while $\mathbf{W}_{i,j}$, $\mathbf{W}_{i,:}$ and $\mathbf{W}_{:,j}$ is the (i, j) -th element, i -th row, j -th column, respectively. We use a single superscript or subscript to denote the index of a matrix, e.g. $\mathbf{X}_i, \mathbf{X}^i$ denote the i -th matrix in a matrices sequence. We use σ and $\bar{\sigma}$ for softmax and hardmax operators, respectively. We use $\text{ReLU}(\mathbf{v}) = \max(\mathbf{v}, \mathbf{0})$ to denote the ReLU activation function where $\max(\cdot)$ function is applied entry-wise to a vector. We use $\text{Cone}(\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_m)$ to denote a convex cone where $\text{Cone}(\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_m) = \{\mathbf{x} : \mathbf{x} = \sum_{i=1}^m a_i \mathbf{a}_i, a_i > 0\}$. We also define the minus operation between a set S and a vector $\mathbf{v}$ as $S - \mathbf{v} = \{\mathbf{x} - \mathbf{v} : \mathbf{x} \in S\}$. In Section 4, we use $[a : b : c]$ to denote a grid $\{a, a + b, a + 2b, \dots, c - b\}$ from a to c , with an interval b .

Transformer networks [Vaswani et al., 2017] are a stack of multiple transformer layers, composed subsequently. A transformer layer has two key components: an attention layer and a token-wise MLP layer, with residual connections around both blocks. We consider the input and output to be sequences of tokens $\mathbf{X} \in \mathbb{R}^{d \times m}$ and $\mathbf{Y} \in \mathbb{R}^{d \times m}$, where m is the number of tokens in the sequence and d is the token dimension.

Definition 1 (Attention Layer). *We define an h -head attention layer parameterized with $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o$ between a single token $\mathbf{x}$ and a token sequence $\mathbf{X}$ as*

$$Att(\mathbf{x}, \mathbf{X}) = \sum_{i=1}^h \mathbf{W}_o^i \mathbf{W}_v^i \mathbf{x} \cdot \sigma((\mathbf{W}_k^i \mathbf{X})^\top \mathbf{W}_q^i \mathbf{x}). \quad (1)$$

The normalizing factor of $\frac{1}{\sqrt{d_{kq}}}$ is subsumed in the weight matrices $\mathbf{W}_k^i$ for notational simplicity.

We can then define the cross attention between two sequences $\mathbf{X}_1 \in \mathbb{R}^{d \times m_1}$ and $\mathbf{X}_2 \in \mathbb{R}^{d \times m_2}$ (We use $\mathbf{x}_k = (\mathbf{X}_1)_{:,k}$ for simplicity):

$$Att(\mathbf{X}_1, \mathbf{X}_2) = [Att(\mathbf{x}_1, \mathbf{X}_2), Att(\mathbf{x}_2, \mathbf{X}_2), \dots, Att(\mathbf{x}_{m_1}, \mathbf{X}_2)].$$

Definition 2 (Standard Transformer Layer). *With definition 1, we define a standard transformer layer τ as*

$$MLP(\mathbf{X}) = [\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{X}_{:,1} + \mathbf{b}_1) + \mathbf{b}_2 + \mathbf{X}_{:,1}, \dots, \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{X}_{:,n} + \mathbf{b}_1) + \mathbf{b}_2 + \mathbf{X}_{:,n}] \quad (2)$$

$$\tau(\mathbf{X}) = MLP(Att(\mathbf{X}, \mathbf{X}) + \mathbf{X}). \quad (3)$$

The definition here omits the layer normalization block for simplicity (following [Kim et al., 2021]).

We denote the set of transformer networks with h heads of size s and r MLP hidden neurons with $\mathcal{T}^{h,s,r}$. In Section 4, we utilize a modified transformer network with hardmax operation $\bar{\sigma}$ instead of softmax σ . We denote this modified version of transformer networks as $\bar{\mathcal{T}}^{h,s,r}$.